
Chapter 11 Inheritance

Topics

11.1 Introduction to Inheritance 

11.2 Polymorphism 

11.1 Introduction to Inheritance

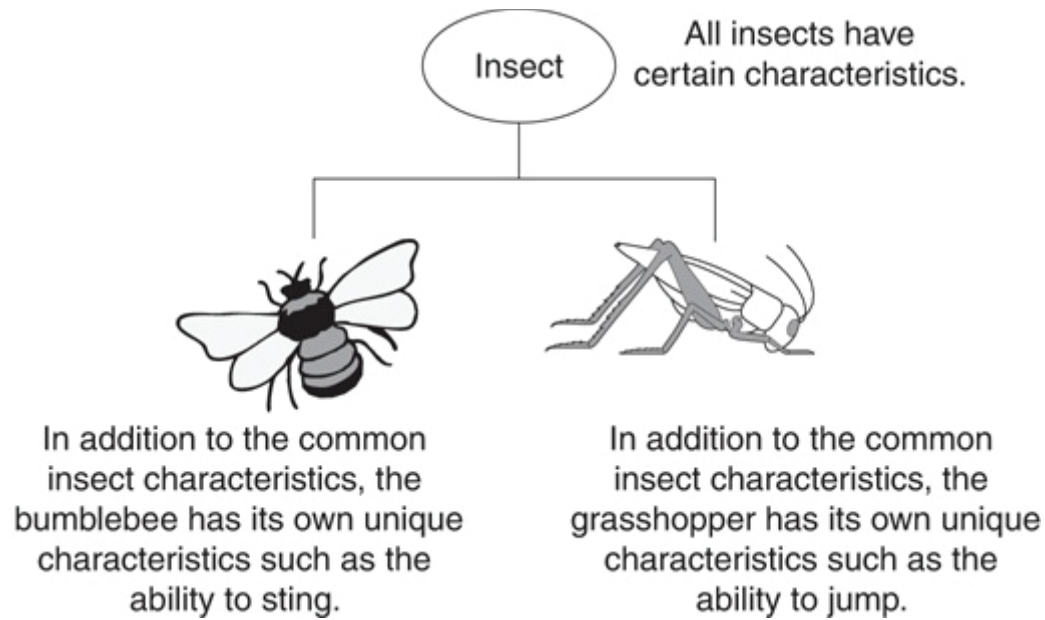
Concept:

Inheritance allows a new class to extend an existing class. The new class inherits the members of the class it extends.

Generalization and Specialization

In the real world, you can find many objects that are specialized versions of other more general objects. For example, the term “insect” describes a general type of creature with various characteristics. Because grasshoppers and bumblebees are insects, they have all the general characteristics of an insect. In addition, they have special characteristics of their own. For example, the grasshopper has its jumping ability, and the bumblebee has its stinger. Grasshoppers and bumblebees are specialized versions of an insect. This is illustrated in **Figure 11-1** .

Figure 11-1 Bumblebees and grasshoppers are specialized versions of an insect



Inheritance and the “Is a” Relationship

When one object is a specialized version of another object, there is an “is a” relationship between them. For example, a grasshopper is an insect. Here are a few other examples of the “is a” relationship:

- A poodle is a dog.
- A car is a vehicle.
- A flower is a plant.
- A rectangle is a shape.
- A football player is an athlete.

When an “is a” relationship exists between objects, it means that the specialized object has all of the characteristics of the general object,

plus additional characteristics that make it special. In object-oriented programming, inheritance is used to create an “is a” relationship among classes. This allows you to extend the capabilities of a class by creating another class that is a specialized version of it.

Inheritance involves a superclass and a subclass. The *superclass* is the general class and the *subclass* is the specialized class. You can think of the subclass as an extended version of the superclass. The subclass inherits attributes and methods from the superclass without any of them having to be rewritten. Furthermore, new attributes and methods may be added to the subclass, and that is what makes it a specialized version of the superclass.



Note:

Superclasses are also called *base classes*, and subclasses are also called *derived classes*. Either set of terms is correct. For consistency, this text will use the terms superclass and subclass.

Let’s look at an example of how inheritance can be used. Suppose we are developing a program that a car dealership can use to manage its inventory of used cars. The dealership’s inventory includes three types of automobiles: cars, pickup trucks, and sport-utility vehicles (SUVs).

Regardless of the type, the dealership keeps the following data about each automobile:

- Make
- Year model
- Mileage
- Price

Each type of vehicle that is kept in inventory has these general characteristics, plus its own specialized characteristics. For cars, the dealership keeps the following additional data:

- Number of doors (2 or 4)

For pickup trucks, the dealership keeps the following additional data:

- Drive type (two-wheel drive or four-wheel drive)

And for SUVs, the dealership keeps the following additional data:

- Passenger capacity

In designing this program, one approach would be to write the following three classes:

- A `Car` class with data attributes for the make, year model, mileage, price, and the number of doors.

- A `Truck` class with data attributes for the make, year model, mileage, price, and the drive type.
- An `SUV` class with data attributes for the make, year model, mileage, price, and the passenger capacity.

This would be an inefficient approach, however, because all three of the classes have a large number of common data attributes. As a result, the classes would contain a lot of duplicated code. In addition, if we discover later that we need to add more common attributes, we would have to modify all three classes.

A better approach would be to write an `Automobile` superclass to hold all the general data about an automobile, then write subclasses for each specific type of automobile. **Program 11-1**  shows the `Automobile` class's code, which appears in a module named `vehicles`.

Program 11-1 *(Lines 1 through 44 of `vehicles.py`)*

```
1  # The Automobile class holds general data
2  # about an automobile in inventory.
3
4  class Automobile:
5      # The __init__ method accepts arguments for the
6      # make, model, mileage, and price. It initializes
7      # the data attributes with these values.
8
```

```
9     def __init__(self, make, model, mileage, price):
10         self.__make = make
11         self.__model = model
12         self.__mileage = mileage
13         self.__price = price
14
15     # The following methods are mutators for the
16     # class's data attributes.
17
18     def set_make(self, make):
19         self.__make = make
20
21     def set_model(self, model):
22         self.__model = model
23
24     def set_mileage(self, mileage):
25         self.__mileage = mileage
26
27     def set_price(self, price):
28         self.__price = price
29
30     # The following methods are the accessors
31     # for the class's data attributes.
32
33     def get_make(self):
34         return self.__make
35
36     def get_model(self):
```

```
37         return self.__model
38
39     def get_mileage(self):
40         return self.__mileage
41
42     def get_price(self):
43         return self.__price
44
```

The `Automobile` class's `__init__` method accepts arguments for the vehicle's make, model, mileage, and price. It uses those values to initialize the following data attributes:

- `__make`
- `__model`
- `__mileage`
- `__price`

(Recall from [Chapter 10](#)  that a data attribute becomes hidden when its name begins with two underscores.) The methods that appear in lines 18 through 28 are mutators for each of the data attributes, and the methods in lines 33 through 43 are the accessors.

The `Automobile` class is a complete class from which we can create objects. If we wish, we can write a program that imports the `vehicle` module and creates instances of the `Automobile` class. However, the `Automobile` class holds only general data about an automobile. It does

not hold any of the specific pieces of data that the dealership wants to keep about cars, pickup trucks, and SUVs. To hold data about those specific types of automobiles, we will write subclasses that inherit from the `Automobile` class. **Program 11-2**  shows the code for the `Car` class, which is also in the `vehicles` module.

Program 11-2 *(Lines 45 through 72 of `vehicles.py`)*

```
45 # The Car class represents a car. It is a subclass
46 # of the Automobile class.
47
48 class Car(Automobile):
49     # The __init__ method accepts arguments for the
50     # car's make, model, mileage, price, and doors.
51
52     def __init__(self, make, model, mileage, price,
53         doors):
54         # Call the superclass's __init__ method and pass
55         # the required arguments. Note that we also have
56         # to pass self as an argument.
57         Automobile.__init__(self, make, model, mileage,
58             price)
59         # Initialize the __doors attribute.
60         self.__doors = doors
```

```
61     # The set_doors method is the mutator for the
62     # __doors attribute.
63
64     def set_doors(self, doors):
65         self.__doors = doors
66
67     # The get_doors method is the accessor for the
68     # __doors attribute.
69
70     def get_doors(self):
71         return self.__doors
72
```

Take a closer look at the first line of the class declaration, in line 48:

```
class Car(Automobile):
```

This line indicates that we are defining a class named `Car`, and it inherits from the `Automobile` class. The `Car` class is the subclass, and the `Automobile` class is the superclass. If we want to express the relationship between the `Car` class and the `Automobile` class, we can say that a `Car` is an `Automobile`. Because the `Car` class extends the `Automobile` class, it inherits all of the methods and data attributes of the `Automobile` class.

Look at the header for the `__init__` method in line 52:

```
def __init__(self, make, model, mileage, price, doors):
```

Notice in addition to the required `self` parameter, the method has parameters named `make`, `model`, `mileage`, `price`, and `doors`. This makes sense because a `Car` object will have data attributes for the car's make, model, mileage, price, and number of doors. Some of these attributes are created by the `Automobile` class, however, so we need to call the `Automobile` class's `__init__` method and pass those values to it. That happens in line 56:

```
Automobile.__init__(self, make, model, mileage, price)
```

This statement calls the `Automobile` class's `__init__` method. Notice the statement passes the `self` variable, as well as the `make`, `model`, `mileage`, and `price` variables as arguments. When that method executes, it initializes the `__make`, `__model`, `__mileage`, and `__price` data attributes. Then, in line 59, the `__doors` attribute is initialized with the value passed into the `doors` parameter:

```
self.__doors = doors
```

The `set_doors` method, in lines 64 through 65, is the mutator for the `__doors` attribute, and the `get_doors` method, in lines 70 through 71, is the accessor for the `__doors` attribute. Before going any further, let's demonstrate the `Car` class, as shown in [Program 11-3](#) .

Program 11-3 (`car_demo.py`)

```
1  # This program demonstrates the Car class.
2
3  import vehicles
4
5  def main():
6      # Create an object from the Car class.
7      # The car is a 2007 Audi with 12,500 miles, priced
8      # at $21,500.00, and has 4 doors.
9      used_car = vehicles.Car('Audi', 2007, 12500, 21500.0,
10
11
12      # Display the car's data.
13      print('Make:', used_car.get_make())
14      print('Model:', used_car.get_model())
15      print('Mileage:', used_car.get_mileage())
16      print('Price:', used_car.get_price())
17      print('Number of doors:', used_car.get_doors())
18
19  # Call the main function.
20  main()
```

Program Output

```
Make: Audi
Model: 2007
Mileage: 12500
Price: 21500.0
Number of doors: 4
```

Line 3 imports the `vehicles` module, which contains the class definitions for the `Automobile` and `Car` classes. Line 9 creates an instance of the `Car` class, passing `'Audi'` as the car's make, 2007 as the car's model, 12,500 as the mileage, 21,500.0 as the car's price, and 4 as the number of doors. The resulting object is assigned to the `used_car` variable.

The statements in lines 12 through 15 call the object's `get_make`, `get_model`, `get_mileage`, and `get_price` methods. Even though the `Car` class does not have any of these methods, it inherits them from the `Automobile` class. Line 16 calls the `get_doors` method, which is defined in the `Car` class.

Now let's look at the `Truck` class, which also inherits from the `Automobile` class. The code for the `Truck` class, which is also in the `vehicles` module, is shown in [Program 11-4](#) .

Program 11-4 (Lines 73 through 100 of *vehicles.py*)

```
73  # The Truck class represents a pickup truck. It is a
74  # subclass of the Automobile class.
75
76  class Truck(Automobile):
77      # The __init__ method accepts arguments for the
78      # Truck's make, model, mileage, price, and drive
79      # type.
80      def __init__(self, make, model, mileage, price,
81                  drive_type):
82          # Call the superclass's __init__ method and pass
83          # the required arguments. Note that we also have
84          # to pass self as an argument.
85          Automobile.__init__(self, make, model, mileage,
86                              price)
87          # Initialize the __drive_type attribute.
88          self.__drive_type = drive_type
89
90      # The set_drive_type method is the mutator for the
91      # __drive_type attribute.
92      def set_drive_type(self, drive_type):
93          self.__drive = drive_type
94
```

```

95     # The get_drive_type method is the accessor for the
96     # __drive_type attribute.
97
98     def get_drive_type(self):
99         return self.__drive_type
100

```

The `Truck` class's `__init__` method begins in line 80. Notice it takes arguments for the truck's make, model, mileage, price, and drive type. Just as the `Car` class did, the `Truck` class calls the `Automobile` class's `__init__` method (in line 84) passing the make, model, mileage, and price as arguments. Line 87 creates the `__drive_type` attribute, initializing it to the value of the `drive_type` parameter.

The `set_drive_type` method in lines 92 through 93 is the mutator for the `__drive_type` attribute, and the `get_drive_type` method in lines 98 through 99 is the accessor for the attribute.

Now let's look at the `SUV` class, which also inherits from the `Automobile` class. The code for the `SUV` class, which is also in the `vehicles` module, is shown in [Program 11-5](#) .

Program 11-5 *(Lines 101 through 128 of `vehicles.py`)*

```

101 # The SUV class represents a sport utility vehicle. It

```

```
102 # is a subclass of the Automobile class.
103
104 class SUV(Automobile):
105     # The __init__ method accepts arguments for the
106     # SUV's make, model, mileage, price, and passenger
107     # capacity.
108
109     def __init__(self, make, model, mileage, price,
pass_cap):
110         # Call the superclass's __init__ method and pass
111         # the required arguments. Note that we also have
112         # to pass self as an argument.
113         Automobile.__init__(self, make, model, mileage,
price)
114
115         # Initialize the __pass_cap attribute.
116         self.__pass_cap = pass_cap
117
118     # The set_pass_cap method is the mutator for the
119     # __pass_cap attribute.
120
121     def set_pass_cap(self, pass_cap):
122         self.__pass_cap = pass_cap
123
124     # The get_pass_cap method is the accessor for the
125     # __pass_cap attribute.
126
127     def get_pass_cap(self):
```



```
128         return self.__pass_cap
```

The `SUV` class's `__init__` method begins in line 109. It takes arguments for the vehicle's make, model, mileage, price, and passenger capacity. Just as the `Car` and `Truck` classes did, the `SUV` class calls the `Automobile` class's `__init__` method (in line 113) passing the make, model, mileage, and price as arguments. Line 116 creates the `__pass_cap` attribute, initializing it to the value of the `pass_cap` parameter.

The `set_pass_cap` method in lines 121 through 122 is the mutator for the `__pass_cap` attribute, and the `get_pass_cap` method in lines 127 through 128 is the accessor for the attribute.

Program 11-6  demonstrates each of the classes we have discussed so far. It creates a `Car` object, a `Truck` object, and an `SUV` object.

Program 11-6 (`car_truck_suv_demo.py`)

```
1  # This program creates a Car object, a Truck object,  
2  # and an SUV object.  
3  
4  import vehicles  
5  
6  def main():
```

```
7      # Create a Car object for a used 2001 BMW
8      # with 70,000 miles, priced at $15,000, with
9      # 4 doors.
10     car = vehicles.Car('BMW', 2001, 70000, 15000.0, 4)
11
12     # Create a Truck object for a used 2002
13     # Toyota pickup with 40,000 miles, priced
14     # at $12,000, with 4-wheel drive.
15     truck = vehicles.Truck('Toyota', 2002, 40000, 12000.0,
16                             '4WD')
17
18     # Create an SUV object for a used 2000
19     # Volvo with 30,000 miles, priced
20     # at $18,500, with 5 passenger capacity.
21     suv = vehicles.SUV('Volvo', 2000, 30000, 18500.0, 5)
22
23     print('USED CAR INVENTORY')
24     print('=====')
25
26     # Display the car's data.
27     print('The following car is in inventory:')
28     print('Make:', car.get_make())
29     print('Model:', car.get_model())
30     print('Mileage:', car.get_mileage())
31     print('Price:', car.get_price())
32     print('Number of doors:', car.get_doors())
33     print()
```

```

34     # Display the truck's data.
35     print('The following pickup truck is in inventory.')
36     print('Make:', truck.get_make())
37     print('Model:', truck.get_model())
38     print('Mileage:', truck.get_mileage())
39     print('Price:', truck.get_price())
40     print('Drive type:', truck.get_drive_type())
41     print()
42
43     # Display the SUV's data.
44     print('The following SUV is in inventory.')
45     print('Make:', suv.get_make())
46     print('Model:', suv.get_model())
47     print('Mileage:', suv.get_mileage())
48     print('Price:', suv.get_price())
49     print('Passenger Capacity:', suv.get_pass_cap())
50
51     # Call the main function.
52     main()

```

Program Output

```

USED CAR INVENTORY
=====

The following car is in inventory:

Make: BMW

Model: 2001

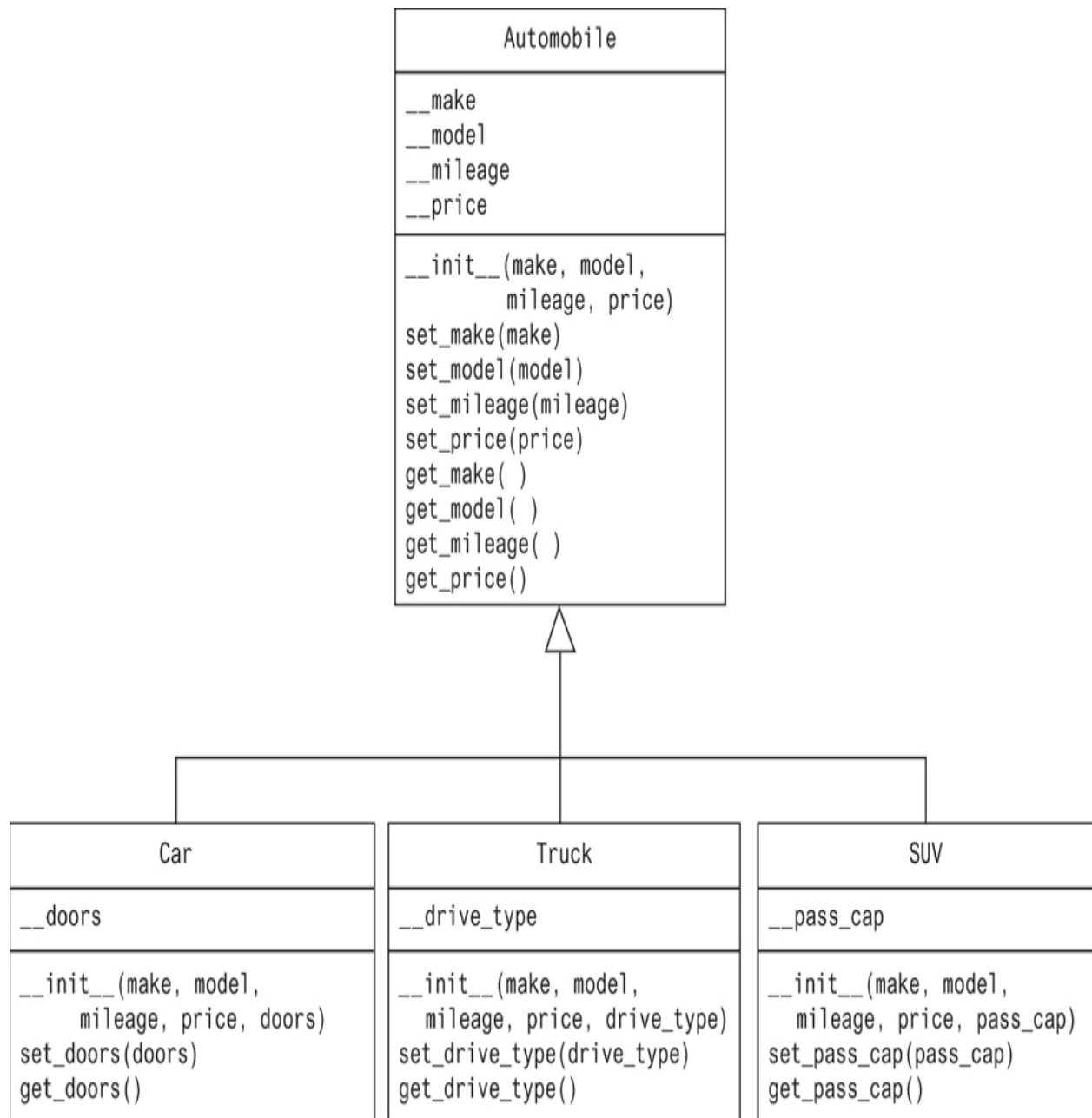
```

```
Mileage: 70000
Price: 15000.0
Number of doors: 4
The following pickup truck is in inventory.
Make: Toyota
Model: 2002
Mileage: 40000
Price: 12000.0
Drive type: 4WD
The following SUV is in inventory.
Make: Volvo
Model: 2000
Mileage: 30000
Price: 18500.0
Passenger Capacity: 5
```

Inheritance in UML Diagrams

You show inheritance in a UML diagram by drawing a line with an open arrowhead from the subclass to the superclass. (The arrowhead points to the superclass.) **Figure 11-2**  is a UML diagram showing the relationship between the `Automobile`, `Car`, `Truck`, and `SUV` classes.

Figure 11-2 UML diagram showing inheritance



In the

Spotlight: Using Inheritance

Bank Financial Systems, Inc. develops financial software for banks and credit unions. The company is developing a new

object-oriented system that manages customer accounts. One of your tasks is to develop a class that represents a savings account. The data that must be held by an object of this class is:

- The account number.
- The interest rate.
- The account balance.

You must also develop a class that represents a certificate of deposit (CD) account. The data that must be held by an object of this class is:

- The account number.
- The interest rate.
- The account balance.
- The account maturity date.

As you analyze these requirements, you realize that a CD account is really a specialized version of a savings account. The class that represents a CD will hold all of the same data as the class that represents a savings account, plus an extra attribute for the maturity date. You decide to design a `SavingsAccount` class to represent a savings account, then design a subclass of `SavingsAccount` named `CD` to represent a CD account. You will store both of these classes in a module named `accounts`. **Program 11-7**  shows the code for the `SavingsAccount` class.

Program 11-7 (Lines 1 through 37 of `accounts.py`)

```
1  # The SavingsAccount class represents a
2  # savings account.
3
4  class SavingsAccount:
5
6      # The __init__ method accepts arguments for the
7      # account number, interest rate, and balance.
8
9      def __init__(self, account_num, int_rate, bal):
10         self.__account_num = account_num
11         self.__interest_rate = int_rate
12         self.__balance = bal
13
14         # The following methods are mutators for the
15         # data attributes.
16
17         def set_account_num(self, account_num):
18             self.__account_num = account_num
19
20         def set_interest_rate(self, int_rate):
21             self.__interest_rate = int_rate
22
23         def set_balance(self, bal):
```

```

24         self.__balance = bal
25
26     # The following methods are accessors for the
27     # data attributes.
28
29     def get_account_num(self):
30         return self.__account_num
31
32     def get_interest_rate(self):
33         return self.__interest_rate
34
35     def get_balance(self):
36         return self.__balance
37

```

The class's `__init__` method appears in lines 9 through 12.

The `__init__` method accepts arguments for the account number, interest rate, and balance. These arguments are used to initialize data attributes named `__account_num`, `__interest_rate`, and `__balance`.

The `set_account_num`, `set_interest_rate`, and `set_balance` methods that appear in lines 17 through 24 are mutators for the data attributes. The `get_account_num`, `get_interest_rate`, and `get_balance` methods that appear in lines 29 through 36 are accessors.

The `CD` class is shown in the next part of **Program 11-7** .

Program 11-7 (Lines 38 through 65 of accounts.py)

```
38 # The CD account represents a certificate of
39 # deposit (CD) account. It is a subclass of
40 # the SavingsAccount class.
41
42 class CD(SavingsAccount):
43
44     # The init method accepts arguments for the
45     # account number, interest rate, balance, and
46     # maturity date.
47
48     def __init__(self, account_num, int_rate, bal,
49 mat_date):
50
51         # Call the superclass __init__ method.
52         SavingsAccount.__init__(self, account_num,
53 int_rate, bal)
54
55         # Initialize the __maturity_date attribute.
56         self.__maturity_date = mat_date
57
58     # The set_maturity_date is a mutator for the
59     # __maturity_date attribute.
60
61     def set_maturity_date(self, mat_date):
```

```

59         self.__maturity_date = mat_date
60
61         # The get_maturity_date method is an accessor
62         # for the __maturity_date attribute.
63
64     def get_maturity_date(self):
65         return self.__maturity_date

```

The `CD` class's `__init__` method appears in lines 48 through 53. It accepts arguments for the account number, interest rate, balance, and maturity date. Line 50 calls the `SavingsAccount` class's `__init__` method, passing the arguments for the account number, interest rate, and balance. After the `SavingsAccount` class's `__init__` method executes, the `__account_num`, `__interest_rate`, and `__balance` attributes will be created and initialized. Then the statement in line 53 creates the `__maturity_date` attribute.

The `set_maturity_date` method in lines 58 through 59 is the mutator for the `__maturity_date` attribute, and the `get_maturity_date` method in lines 64 through 65 is the accessor.

To test the classes, we use the code shown in **Program 11-8** . This program creates an instance of the `SavingsAccount` class to represent a savings account, and an instance of the `CD` class to represent a certificate of deposit account.

Program 11-8 `(account_demo.py)`

```
1  # This program creates an instance of the
SavingsAccount
2  # class and an instance of the CD account.
3
4  import accounts
5
6  def main():
7      # Get the account number, interest rate,
8      # and account balance for a savings account.
9      print('Enter the following data for a savings
account.')
10     acct_num = input('Account number: ')
11     int_rate = float(input('Interest rate: '))
12     balance = float(input('Balance: '))
13
14     # Create a SavingsAccount object.
15     savings = accounts.SavingsAccount(acct_num,
int_rate,
16                                     balance)
17
18     # Get the account number, interest rate,
19     # account balance, and maturity date for a CD.
20     print('Enter the following data for a CD.')
21     acct_num = input('Account number: ')
22     int_rate = float(input('Interest rate: '))
```

```
23     balance = float(input('Balance: '))
24     maturity = input('Maturity date: ')
25
26     # Create a CD object.
27     cd = accounts.CD(acct_num, int_rate, balance,
maturity)
28
29     # Display the data entered.
30     print('Here is the data you entered:')
31     print()
32     print('Savings Account')
33     print('-----')
34     print('Account number:',
savings.get_account_num())
35     print('Interest rate:',
savings.get_interest_rate())
36     print('Balance: $',
37           format(savings.get_balance(), ',.2f'),
38           sep='')
39     print()
40     print('CD')
41     print('-----')
42     print('Account number:', cd.get_account_num())
43     print('Interest rate:', cd.get_interest_rate())
44     print('Balance: $',
45           format(cd.get_balance(), ',.2f'),
46           sep='')
47     print('Maturity date:', cd.get_maturity_date())
```

```
48
49 # Call the main function.
50 main()
```

Program Output (with input shown in bold)

Enter the following data for a savings account.

Account number: **1234SA**

Interest rate: **3.5**

Balance: **1000.00**

Enter the following data for a CD.

Account number: **2345CD**

Interest rate: **5.6**

Balance: **2500.00**

Maturity date: **12/12/2019**

Here is the data you entered:

Savings Account

Account number: 1234SA

Interest rate: 3.5

Balance: \$1,000.00

CD

Account number: 2345CD

Interest rate: 5.6

Balance: \$2,500.00

Maturity date: 12/12/2019



Checkpoint

11.1 In this section, we discussed superclasses and subclasses. Which is the general class, and which is the specialized class?

11.2 What does it mean to say there is an “is a” relationship between two objects?

11.3 What does a subclass inherit from its superclass?

11.4 Look at the following code, which is the first line of a class definition. What is the name of the superclass? What is the name of the subclass?

```
class Canary(Bird):
```

11.2 Polymorphism

Concept:

Polymorphism allows subclasses to have methods with the same names as methods in their superclasses. It gives the ability for a program to call the correct method depending on the type of object that is used to call it.

The term *polymorphism* refers to an object's ability to take different forms. It is a powerful feature of object-oriented programming. In this section, we will look at two essential ingredients of polymorphic behavior:

1. The ability to define a method in a superclass, then define a method with the same name in a subclass. When a subclass method has the same name as a superclass method, it is often said that the subclass method *overrides* the superclass method.
2. The ability to call the correct version of an overridden method, depending on the type of object that is used to call it. If a subclass object is used to call an overridden method, then the subclass's version of the method is the one that will execute. If

a superclass object is used to call an overridden method, then the superclass's version of the method is the one that will execute.

Actually, you've already seen method overriding at work. Each subclass that we have examined in this chapter has a method named `__init__` that overrides the superclass's `__init__` method. When an instance of the subclass is created, it is the subclass's `__init__` method that automatically gets called.

Method overriding works for other class methods too. Perhaps the best way to describe polymorphism is to demonstrate it, so let's look at a simple example. **Program 11-9**  shows the code for a class named `Mammal`, which is in a module named `animals`.

Program 11-9 (Lines 1 through 22 of `animals.py`)

```
1  # The Mammal class represents a generic mammal.
2
3  class Mammal:
4
5      # The __init__ method accepts an argument for
6      # the mammal's species.
7
8      def __init__(self, species):
9          self.__species = species
```



```

10
11     # The show_species method displays a message
12     # indicating the mammal's species.
13
14     def show_species(self):
15         print('I am a', self.__species)
16
17     # The make_sound method is the mammal's
18     # way of making a generic sound.
19
20     def make_sound(self):
21         print('Grrrrr')
22

```

The `Mammal` class has three methods: `__init__`, `show_species` and `make_sound`. Here is an example of code that creates an instance of the class and calls these methods:

```

import animals

mammal = animals.Mammal('regular mammal')

mammal.show_species()

mammal.make_sound()

```

This code will display the following:

```

I am a regular mammal

```

```
Grrrrr
```

The next part of **Program 11-9**  shows the `Dog` class. The `Dog` class, which is also in the `animals` module, is a subclass of the `Mammal` class.

Program 11-9 (Lines 23 through 38 of `animals.py`)

```
23 # The Dog class is a subclass of the Mammal class.
24
25 class Dog(Mammal):
26
27     # The __init__ method calls the superclass's
28     # __init__ method passing 'Dog' as the species.
29
30     def __init__(self):
31         Mammal.__init__(self, 'Dog')
32
33     # The make_sound method overrides the superclass's
34     # make_sound method.
35
36     def make_sound(self):
37         print('Woof! Woof!')
38
```

Even though the `Dog` class inherits the `__init__` and `make_sound` methods that are in the `Mammal` class, those methods are not adequate for the `Dog` class. So, the `Dog` class has its own `__init__` and `make_sound` methods, which perform actions that are more appropriate for a dog. We say that the `__init__` and `make_sound` methods in the `Dog` class override the `__init__` and `make_sound` methods in the `Mammal` class. Here is an example of code that creates an instance of the `Dog` class and calls the methods:

```
import animals
dog = animals.Dog()
dog.show_species()
dog.make_sound()
```

This code will display the following:

```
I am a Dog
Woof! Woof!
```

When we use a `Dog` object to call the `show_species` and `make_sound` methods, the versions of these methods that are in the `Dog` class are the ones that execute. Next, look at [Program 11-10](#), which shows the `Cat` class. The `Cat` class, which is also in the `animals` module, is another subclass of the `Mammal` class.

Program 11-9 (Lines 39 through 53 of `animals.py`)

```
39 # The Cat class is a subclass of the Mammal class.
40
41 class Cat(Mammal):
42
43     # The __init__ method calls the superclass's
44     # __init__ method passing 'Cat' as the species.
45
46     def __init__(self):
47         Mammal.__init__(self, 'Cat')
48
49     # The make_sound method overrides the superclass's
50     # make_sound method.
51
52     def make_sound(self):
53         print('Meow')
```

The `Cat` class also overrides the `Mammal` class's `__init__` and `make_sound` methods. Here is an example of code that creates an instance of the `Cat` class and calls these methods:

```
import animals
cat = animals.Cat()
cat.show_species()
```

```
cat.make_sound()
```

This code will display the following:

```
I am a Cat
```

```
Meow
```

When we use a `Cat` object to call the `show_species` and `make_sound` methods, the versions of these methods that are in the `Cat` class are the ones that execute.

The `isinstance` Function

Polymorphism gives us a great deal of flexibility when designing programs. For example, look at the following function:

```
def show_mammal_info(creature):  
    creature.show_species()  
    creature.make_sound()
```

We can pass any object as an argument to this function, and as long as it has a `show_species` method and a `make_sound` method, the function will call those methods. In essence, we can pass any object

that “is a” `Mammal` (or a subclass of `Mammal`) to the function. **Program 11-10**  demonstrates.

Program 11-10 *(polymorphism_demo.py)*

```
1  # This program demonstrates polymorphism.
2
3  import animals
4
5  def main():
6      # Create a Mammal object, a Dog object, and
7      # a Cat object.
8      mammal = animals.Mammal('regular animal')
9      dog = animals.Dog()
10     cat = animals.Cat()
11
12     # Display information about each one.
13     print('Here are some animals and')
14     print('the sounds they make.')
15     print('-----')
16     show_mammal_info(mammal)
17     print()
18     show_mammal_info(dog)
19     print()
20     show_mammal_info(cat)
21
22     # The show_mammal_info function accepts an object
23     # as an argument, and calls its show_species
```

```
24 # and make_sound methods.
25
26 def show_mammal_info(creature):
27     creature.show_species()
28     creature.make_sound()
29
30 # Call the main function.
31 main()
```

Program Output

```
Here are some animals and
the sounds they make.
-----
I am a regular animal
Grrrrr
I am a Dog
Woof! Woof!
I am a Cat
Meow
```

But what happens if we pass an object that is not a `Mammal`, and not of a subclass of `Mammal` to the function? For example, what will happen when **Program 11-11**  runs?

Program 11-11 `(wrong_type.py)`

```
1 def main():
2     # Pass a string to show_mammal_info ...
3     show_mammal_info('I am a string')
4
5     # The show_mammal_info function accepts an object
6     # as an argument, and calls its show_species
7     # and make_sound methods.
8
9     def show_mammal_info(creature):
10         creature.show_species()
11         creature.make_sound()
12
13     # Call the main function.
14     main()
```

In line 3, we call the `show_mammal_info` function passing a string as an argument. When the interpreter attempts to execute line 10, however, an `AttributeError` exception will be raised because strings do not have a method named `show_species`.

We can prevent this exception from occurring by using the built-in function `isinstance`. You can use the `isinstance` function to determine whether an object is an instance of a specific class, or a subclass of that class. Here is the general format of the function call:

```
isinstance(object, ClassName)
```


In the general format, `object` is a reference to an object, and `ClassName` is the name of a class. If the object referenced by `object` is an instance of `ClassName` or is an instance of a subclass of `ClassName`, the function returns true. Otherwise, it returns false. **Program 11-12**  shows how we can use it in the `show_mammal_info` function.

Program 11-12 (polymorphism_demo2.py)

```
1  # This program demonstrates polymorphism.
2
3  import animals
4
5  def main():
6      # Create an Mammal object, a Dog object, and
7      # a Cat object.
8      mammal = animals.Mammal('regular animal')
9      dog = animals.Dog()
10     cat = animals.Cat()
11
12     # Display information about each one.
13     print('Here are some animals and')
14     print('the sounds they make.')
15     print('-----')
16     show_mammal_info(mammal)
17     print()
18     show_mammal_info(dog)
```

```
19     print()
20     show_mammal_info(cat)
21     print()
22     show_mammal_info('I am a string')
23
24     # The show_mammal_info function accepts an object
25     # as an argument and calls its show_species
26     # and make_sound methods.
27
28     def show_mammal_info(creature):
29         if isinstance(creature, animals.Mammal):
30             creature.show_species()
31             creature.make_sound()
32         else:
33             print('That is not a Mammal!')
34
35     # Call the main function.
36     main()
```

Program Output

```
Here are some animals and
the sounds they make.
-----
I am a regular animal
Grrrrr
I am a Dog
```

```
Woof! Woof!  
  
I am a Cat  
  
Meow  
  
That is not a Mammal!
```

In lines 16, 18, and 20 we call the `show_mammal_info` function, passing references to a `Mammal` object, a `Dog` object, and a `Cat` object. In line 22, however, we call the function and pass a string as an argument. Inside the `show_mammal_info` function, the `if` statement in line 29 calls the `isinstance` function to determine whether the argument is an instance of `Mammal` (or a subclass). If it is not, an error message is displayed.



Checkpoint

11.5 Look at the following class definitions:

```
class Vegetable:  
    def __init__(self, vegtype):  
        self.__vegtype = vegtype  
    def message(self):  
        print("I'm a vegetable.")  
class Potato(Vegetable):  
    def __init__(self):  
        Vegetable.__init__(self, 'potato')  
    def message(self):  
        print("I'm a potato.")
```

Given these class definitions, what will the following statements display?

```
v = Vegetable('veggie')  
p = Potato()  
v.message()  
p.message()
```

Review Questions

Multiple Choice

1. In an inheritance relationship, the _____ is the general class.
 - a. subclass
 - b. superclass
 - c. slave class
 - d. child class

2. In an inheritance relationship, the _____ is the specialized class.
 - a. superclass
 - b. master class
 - c. subclass
 - d. parent class

3. Suppose a program uses two classes: `Airplane` and `JumboJet`. Which of these would most likely be the subclass?
 - a. `Airplane`
 - b. `JumboJet`
 - c. Both
 - d. Neither

4. This characteristic of object-oriented programming allows the correct version of an overridden method to be called when an instance of a subclass is used to call it.
 - a. polymorphism
 - b. inheritance
 - c. generalization
 - d. specialization
5. You can use this to determine whether an object is an instance of a class.
 - a. the `in` operator
 - b. the `is_object_of` function
 - c. the `isinstance` function
 - d. the error messages that are displayed when a program crashes

True or False

1. Polymorphism allows you to write methods in a subclass that have the same name as methods in the superclass.
2. It is not possible to call a superclass's `__init__` method from a subclass's `__init__` method.
3. A subclass can have a method with the same name as a method in the superclass.
4. Only the `__init__` method can be overridden.

5. You cannot use the `isinstance` function to determine whether an object is an instance of a subclass of a class.

Short Answer

1. What does a subclass inherit from its superclass?
2. Look at the following class definition. What is the name of the superclass? What is the name of the subclass?

```
class Tiger(Felis):
```

3. What is an overridden method?

Algorithm Workbench

1. Write the first line of the definition for a `Poodle` class. The class should extend the `Dog` class.
2. Look at the following class definitions:

```
class Plant:
    def __init__(self, plant_type):
        self.__plant_type = plant_type
    def message(self):
        print("I'm a plant.")
class Tree(Plant):
```

```
def __init__(self):  
    Plant.__init__(self, 'tree')  
  
def message(self):  
    print("I'm a tree.")
```

Given these class definitions, what will the following statements display?

```
p = Plant('sapling')  
t = Tree()  
p.message()  
t.message()
```

3. Look at the following class definition:

```
class Beverage:  
    def __init__(self, bev_name):  
        self.__bev_name = bev_name
```

Write the code for a class named `Cola` that is a subclass of the `Beverage` class. The `Cola` class's `__init__` method should call the `Beverage` class's `__init__` method, passing 'cola' as an argument.

Programming Exercises

1. `Employee` and `ProductionWorker` Classes

Write an `Employee` class that keeps data attributes for the following pieces of information:

- Employee name
- Employee number

Next, write a class named `ProductionWorker` that is a subclass of the `Employee` class. The `ProductionWorker` class should keep data attributes for the following information:

- Shift number (an integer, such as 1, 2, or 3)
- Hourly pay rate

The workday is divided into two shifts: day and night. The shift attribute will hold an integer value representing the shift that the employee works. The day shift is shift 1 and the night shift is shift 2. Write the appropriate accessor and mutator methods for each class.

Once you have written the classes, write a program that creates an object of the `ProductionWorker` class and prompts the user to enter data for each of the object's data attributes. Store the data in the object, then use the object's accessor methods to retrieve it and display it on the screen.

2. `ShiftSupervisor` Class

In a particular factory, a shift supervisor is a salaried employee who supervises a shift. In addition to a salary, the shift supervisor earns a yearly bonus when his or her shift meets production goals. Write a `ShiftSupervisor` class that is a subclass of the `Employee` class you created in Programming Exercise 1. The `ShiftSupervisor` class should keep a data attribute for the annual salary, and a data attribute for the annual production bonus that a shift supervisor has earned. Demonstrate the class by writing a program that uses a `ShiftSupervisor` object.

3. `Person` and `Customer` Classes



The `Person` and `Customer` Classes

Write a class named `Person` with data attributes for a person's name, address, and telephone number. Next, write a class named `Customer` that is a subclass of the `Person` class. The `Customer` class should have a data attribute for a customer number, and a Boolean data attribute indicating whether the

customer wishes to be on a mailing list. Demonstrate an instance of the `Customer` class in a simple program.

Chapter 12 Recursion

Topics

12.1 Introduction to Recursion 

12.2 Problem Solving with Recursion 

12.3 Examples of Recursive Algorithms 